

Test-driven development

What is test-driven development (TDD)?

TDD is a software development methodology where tests are written for a feature before writing the code for that feature.

What is test-driven development (TDD)?

TDD is a software development methodology where tests are written for a feature before writing the code for that feature.

Yes that sounds weird, but let's give the computer scientists some credit. We actually do it all the time intuitively.

1. Write a failing test.

The test fails because the code it tests doesn't exist yet or it is not implemented yet.

1. Write a failing test.

The test fails because the code it tests doesn't exist yet or it is not implemented yet.

2. Write just enough code to make the test pass.

Write the minimum amount of code necessary to pass the test. Don't add any additional features or improvements.

1. Write a failing test.

The test fails because the code it tests doesn't exist yet or it is not implemented yet.

2. Write just enough code to make the test pass.

Write the minimum amount of code necessary to pass the test. Don't add any additional features or improvements.

3. Refactor the code as needed.

Improve design, implementation, maintainability, all the while constantly testing to make sure that all tests still pass.

Two kinds of tests

By writing tests first and then writing the code to make the tests pass, you get better code quality, improved bug detection, and a more enjoyable development experience.

Two kinds of tests

By writing tests first and then writing the code to make the tests pass, you get better code quality, improved bug detection, and a more enjoyable development experience.

Unit tests

Two kinds of tests

By writing tests first and then writing the code to make the tests pass, you get better code quality, improved bug detection, and a more enjoyable development experience.

Unit tests

Tests individual pieces of code, such as functions or methods, in isolation.

Two kinds of tests

By writing tests first and then writing the code to make the tests pass, you get better code quality, improved bug detection, and a more enjoyable development experience.

Unit tests

Tests individual pieces of code, such as functions or methods, in isolation.

Integration tests

Two kinds of tests

By writing tests first and then writing the code to make the tests pass, you get better code quality, improved bug detection, and a more enjoyable development experience.

Unit tests

Tests individual pieces of code, such as functions or methods, in isolation.

Integration tests

Tests how multiple pieces of code work together.

- Consider a simple example of a function that performs a statistical calculation (e.g., linear regression).

- Consider a simple example of a function that performs a statistical calculation (e.g., linear regression).
- Before writing any production code, write test cases that test the expected behavior of the function.

- Consider a simple example of a function that performs a statistical calculation (e.g., linear regression).
- Before writing any production code, write test cases that test the expected behavior of the function.
 - Function gives correct output with known inputs.

TDD in practice: An example

- Consider a simple example of a function that performs a statistical calculation (e.g., linear regression).
- Before writing any production code, write test cases that test the expected behavior of the function.
 - Function gives correct output with known inputs.
 - Function correctly verifies input (throwing errors for bad input is desirable behavior!).

TDD in practice: An example

- Consider a simple example of a function that performs a statistical calculation (e.g., linear regression).
- Before writing any production code, write test cases that test the expected behavior of the function.
 - Function gives correct output with known inputs.
 - Function correctly verifies input (throwing errors for bad input is desirable behavior!).
- Write the function, making sure it passes all the test cases.

TDD in practice: An example

- Consider a simple example of a function that performs a statistical calculation (e.g., linear regression).
- Before writing any production code, write test cases that test the expected behavior of the function.
 - Function gives correct output with known inputs.
 - Function correctly verifies input (throwing errors for bad input is desirable behavior!).
- Write the function, making sure it passes all the test cases.
- Refactor the code as needed, making sure all tests still pass.

TDD in practice: An example

- Consider a simple example of a function that performs a statistical calculation (e.g., linear regression).
- Before writing any production code, write test cases that test the expected behavior of the function.
 - Function gives correct output with known inputs.
 - Function correctly verifies input (throwing errors for bad input is desirable behavior!).
- Write the function, making sure it passes all the test cases.
- Refactor the code as needed, making sure all tests still pass.
- Repeat the process for any additional features or bug fixes.

- TDD helps to ensure code correctness and reduce the number of bugs.

- TDD helps to ensure code correctness and reduce the number of bugs.
- It makes it easier to make changes to code and add new features, as any breaking changes will be caught by the tests.

- TDD helps to ensure code correctness and reduce the number of bugs.
- It makes it easier to make changes to code and add new features, as any breaking changes will be caught by the tests.
- TDD improves code maintainability by providing a clear and complete set of tests for the code.

- TDD helps to ensure code correctness and reduce the number of bugs.
- It makes it easier to make changes to code and add new features, as any breaking changes will be caught by the tests.
- TDD improves code maintainability by providing a clear and complete set of tests for the code.
- It also helps to improve the development process by making it more incremental and iterative, allowing for a faster development cycle.

- Cognitive modeling is amateur software development.

- Cognitive modeling is amateur software development.
- Like any software development, it benefits from TDD.

- Cognitive modeling is amateur software development.
- Like any software development, it benefits from TDD.
- Tests verify correctness of a model's predictions and behavior.

TDD in cognitive model development

- Cognitive modeling is amateur software development.
- Like any software development, it benefits from TDD.
- Tests verify correctness of a model's predictions and behavior.
- Once TDD is implemented, we can go through rapid cycles of model development:

TDD in cognitive model development

- Cognitive modeling is amateur software development.
- Like any software development, it benefits from TDD.
- Tests verify correctness of a model's predictions and behavior.
- Once TDD is implemented, we can go through rapid cycles of model development:
 - Make it easier to make changes or add new features (to the cognitive model, not just software).

TDD in cognitive model development

- Cognitive modeling is amateur software development.
- Like any software development, it benefits from TDD.
- Tests verify correctness of a model's predictions and behavior.
- Once TDD is implemented, we can go through rapid cycles of model development:
 - Make it easier to make changes or add new features (to the cognitive model, not just software).
 - Ensure the reliability and robustness of a model.

- Cognitive modeling is amateur software development.
- Like any software development, it benefits from TDD.
- Tests verify correctness of a model's predictions and behavior.
- Once TDD is implemented, we can go through rapid cycles of model development:
 - Make it easier to make changes or add new features (to the cognitive model, not just software).
 - Ensure the reliability and robustness of a model.
 - Continuous testing cycles facilitate rapid model development.

- Comparison of model predictions against observed data: does the model (still) fit? Does it now fit better, or worse?

TDD in cognitive model development

- Comparison of model predictions against observed data: does the model (still) fit? Does it now fit better, or worse?
- Comparison of model predictions against common sense.

TDD in cognitive model development

- Comparison of model predictions against observed data: does the model (still) fit? Does it now fit better, or worse?
- Comparison of model predictions against common sense.
- Testing the model's robustness when we change...

TDD in cognitive model development

- Comparison of model predictions against observed data: does the model (still) fit? Does it now fit better, or worse?
- Comparison of model predictions against common sense.
- Testing the model's robustness when we change...
 - independent variables

TDD in cognitive model development

- Comparison of model predictions against observed data: does the model (still) fit? Does it now fit better, or worse?
- Comparison of model predictions against common sense.
- Testing the model's robustness when we change...
 - independent variables
 - dependent variables

TDD in cognitive model development

- Comparison of model predictions against observed data: does the model (still) fit? Does it now fit better, or worse?
- Comparison of model predictions against common sense.
- Testing the model's robustness when we change...
 - independent variables
 - dependent variables
 - convenience assumptions we may have made

TDD in cognitive model development

- Comparison of model predictions against observed data: does the model (still) fit? Does it now fit better, or worse?
- Comparison of model predictions against common sense.
- Testing the model's robustness when we change...
 - independent variables
 - dependent variables
 - convenience assumptions we may have made
 - structural relationships within the model

TDD in cognitive model development

- Comparison of model predictions against observed data: does the model (still) fit? Does it now fit better, or worse?
- Comparison of model predictions against common sense.
- Testing the model's robustness when we change...
 - independent variables
 - dependent variables
 - convenience assumptions we may have made
 - structural relationships within the model
 - add entirely new features

TDD in cognitive model development

- Comparison of model predictions against observed data: does the model (still) fit? Does it now fit better, or worse?
- Comparison of model predictions against common sense.
- Testing the model's robustness when we change...
 - independent variables
 - dependent variables
 - convenience assumptions we may have made
 - structural relationships within the model
 - add entirely new features
 - ...

TDD in cognitive model development

- Comparison of model predictions against observed data: does the model (still) fit? Does it now fit better, or worse?
- Comparison of model predictions against common sense.
- Testing the model's robustness when we change...
 - independent variables
 - dependent variables
 - convenience assumptions we may have made
 - structural relationships within the model
 - add entirely new features
 - ...
- Often the test can be evaluated informally (e.g., looking at a figure of model vs. data), but ideally it is automated.

1. Write a test: Write a test that defines a desired behavior or feature of the code you are developing.

1. Write a test: Write a test that defines a desired behavior or feature of the code you are developing.
2. Watch it fail: Run the test and see that it fails, as the desired behavior or feature has not been implemented yet.

1. Write a test: Write a test that defines a desired behavior or feature of the code you are developing.
2. Watch it fail: Run the test and see that it fails, as the desired behavior or feature has not been implemented yet.
3. Write the code: Write the minimum amount of code needed to make the test pass.

1. Write a test: Write a test that defines a desired behavior or feature of the code you are developing.
2. Watch it fail: Run the test and see that it fails, as the desired behavior or feature has not been implemented yet.
3. Write the code: Write the minimum amount of code needed to make the test pass.
4. Watch it pass: Run the test and see that it passes, as the desired behavior or feature has now been implemented.

1. Write a test: Write a test that defines a desired behavior or feature of the code you are developing.
2. Watch it fail: Run the test and see that it fails, as the desired behavior or feature has not been implemented yet.
3. Write the code: Write the minimum amount of code needed to make the test pass.
4. Watch it pass: Run the test and see that it passes, as the desired behavior or feature has now been implemented.
5. Refactor: Refactor the code as needed, making sure all tests continue to pass.

1. Repeat the process: Repeat the process of adding a test, running all tests, writing the code, running all tests, and refactoring as needed for each desired behavior or feature.

1. Repeat the process: Repeat the process of adding a test, running all tests, writing the code, running all tests, and refactoring as needed for each desired behavior or feature.
2. Continuously verify: Continuously verify the correctness of the code through the tests.

1. Repeat the process: Repeat the process of adding a test, running all tests, writing the code, running all tests, and refactoring as needed for each desired behavior or feature.
2. Continuously verify: Continuously verify the correctness of the code through the tests.
3. Maintaining a test suite: Maintaining a comprehensive test suite that covers all desired behaviors and features of the code.

```
import unittest

class TestStringMethods(unittest.TestCase):
    def test_upper(self):
        self.assertEqual('foo'.upper(), 'FOO')
    def test_isupper(self):
        self.assertTrue('FOO'.isupper())
        self.assertFalse('Foo'.isupper())
    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', 'world'])
        with self.assertRaises(TypeError):
            s.split(2)
```

```
# Run the tests  
if __name__ == '__main__':  
    unittest.main()
```

```
# Run the tests
```

```
if __name__ == '__main__':  
    unittest.main()
```

```
# If using jupyter...
```

```
if __name__ == '__main__':  
    unittest.main(argv=['first-arg-is-ignored'], exit=False)
```